

TP Socle - Un annuaire, deux approches

Travail : individuel

Prérequis : complexité et notation Big O, C de base (tableaux, pointeurs, `struct`, `malloc`, `free`), listes chaînées

Organisation du TP

Partie	Travail
Mise en place	Créer le dossier de travail et le Makefile
Exercice 0	Le plus petit programme possible
Exercice 1	Analyser le problème avant de coder
Exercice 2	Un tableau qui grandit tout seul
Exercice 3	La recherche séquentielle
Exercice 4	Calculer une adresse plutôt que chercher
Exercice 5	La table de hachage avec chaînage
Exercice 6	Synthèse : un banc de test commun

Les étudiants ayant terminé le TP Socle enchaînent sur le TP Approfondissement.

Méthode de travail

Trois règles à appliquer du début à la fin du TP.

1. **Compilez souvent.** Après chaque fonction écrite, compilez et lancez. Jamais à la fin.
2. **Compilez avec les avertissements.** `gcc -Wall -Wextra -Werror` signale des erreurs que le compilateur accepterait sinon en silence. Un avertissement est une erreur qui ne s'est pas encore manifestée.
3. **Lisez le message d'erreur.** Il indique presque toujours le numéro de la ligne fautive. Le premier message est le seul fiable : les suivants sont souvent des conséquences.

Mise en place

Créez le dossier de travail :

```
mkdir -p ~/annuaire && cd ~/annuaire
git init
```

Ce dépôt servira pendant tout le module, jusqu'au projet final.

Créez le fichier `Makefile` suivant. Attention : les lignes de commande doivent commencer par une **tabulation**, pas par des espaces.

```
CC      = gcc
CFLAGS = -Wall -Wextra -std=c11 -g

test: sequentiel.o hachage.o test.o
    $(CC) $(CFLAGS) -o test sequentiel.o hachage.o test.o

%.o: %.c annuaire.h
    $(CC) $(CFLAGS) -c $<

clean:
    rm -f *.o test
```

Vérifiez que votre compilateur répond : `gcc --version`.

Exercice 0 - Le plus petit programme possible

Élément	Rôle
<code>annuaire.h</code>	Déclare la structure et les fonctions, inclus par tous les fichiers
<code>#ifndef / #define / #endif</code>	Garde d'inclusion : évite que le fichier soit lu deux fois
<code>gcc -Wall -Wextra</code>	Active les avertissements
<code>make</code>	Recompile uniquement ce qui a changé

Consigne

1. Créez `annuaire.h` à partir du modèle ci-dessous, en complétant la structure `User` : une adresse e-mail et un identifiant numérique.
2. Créez `test.c` avec un `main` qui déclare un `User`, lui donne des valeurs (notamment avec `snprintf` pour l'email de l'utilisateur) et les affiche.
3. Compilez avec `gcc -Wall -Wextra -std=c11 -o test test.c`, puis lancez.
4. Remplacez volontairement `snprintf(u.email, EMAIL_MAX, "%s", "alice@mail.com")` par `u.email = "alice@mail.com"`. Recompilez et notez le message d'erreur.
5. Question : pourquoi ne peut-on pas affecter une chaîne à un tableau de `char` avec `=` ?

Modèle à compléter

```

#ifndef ANNUAIRE_H
#define ANNUAIRE_H

#include <stdbool.h>
#define EMAIL_MAX 100

typedef struct {
    _____; /* l'adresse e-mail */
    _____; /* l'identifiant numerique */
} User;

#endif

```

Format attendu

Contenu de `annuaire.h` :

Contenu de `test.c` :

Question	Commande	Résultat observé
3		
4		

Réponse à la question 5 : _____

Exercice 1 - Analyser le problème avant de coder

La grille d'analyse en cinq points, vue en cours :

Point	Question
Entrées	Quelles sont les entrées, et sous quelle forme ?
Sorties	Quelle sortie attend-on exactement ?
Contraintes	Quelles contraintes de temps, de mémoire, de correction ?
Volume	Quel volume de données, et comment évolue-t-il ?
Fréquence	À quelle fréquence chaque opération est-elle appelée ?

Consigne

Le problème : une API d'inscription doit vérifier qu'une adresse e-mail n'est pas déjà utilisée avant de créer un compte.

1. Remplissez la grille en une phrase par point, appliquée à ce problème précis.
2. Question : parmi les cinq points, lesquels vous aideraient à choisir entre deux structures de données ? Lesquels ne vous apprennent rien sur ce choix ?
3. Question : deux annuaires ont les mêmes entrées et la même sortie, l'un contient 30 employés consultés deux fois par jour, l'autre 5 millions de comptes interrogés mille fois par seconde. Quels sont les deux points de la grille qui les distinguent ?

Format attendu

Point	Votre réponse
Entrées	
Sorties	
Contraintes	
Volume	
Fréquence	

Réponse à la question 2 : _____

Réponse à la question 3 : _____

Exercice 2 - Un tableau qui grandit tout seul

Élément	Rôle
<code>malloc(n * sizeof(User))</code>	Alloue de la place pour <code>n</code> utilisateurs
<code>realloc(ptr, n * sizeof(User))</code>	Agrandit une zone déjà allouée, en recopiant si nécessaire
<code>free(ptr)</code>	Libère la zone
<code>static</code> devant une variable globale	La rend invisible depuis les autres fichiers

Trois variables sont nécessaires pour décrire l'annuaire :

Variable	Signification
<code>annuaire</code>	Le tableau lui-même
<code>taille</code>	Le nombre d'utilisateurs réellement présents
<code>capacite</code>	Le nombre de places disponibles

Attention, `realloc` peut échouer et renvoyer `NULL`. Si vous écrivez `annuaire = realloc(annuaire, ...)`, vous perdez alors l'adresse de l'ancienne zone, qui devient irrécupérable. Passez toujours par une variable temporaire.

Consigne

1. Créez `sequentiel.c` et complétez `seq_insert` à partir du modèle.
2. La capacité doit démarrer à 16 et **doubler** chaque fois que le tableau est plein.
3. Écrivez `seq_free` qui libère le tableau et remet les trois variables à zéro.
4. Dans un `main`, insérez 40 utilisateurs et affichez `capacite` après chaque insertion. Notez la suite de valeurs obtenue.
5. Question : combien de fois `realloc` a-t-il été appelé pour ces 40 insertions ? Et pour 1000 insertions ?

Modèle à compléter

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "annuaire.h"

static User *annuaire = NULL;
static int taille = 0;
static int capacite = 0;

void seq_insert(const char *email, int id)
{
    if (taille == _____)
    {
        int nouvelle;

        if (capacite == 0)
        {
            nouvelle = _____;
        }
        else
        {
            nouvelle = _____;
        }

        User *tmp = realloc(
            annuaire,
            (size_t)nouvelle * sizeof(User)
        );

        if (tmp == NULL)
        {
            perror("realloc");
            exit(EXIT_FAILURE);
        }

        annuaire = tmp;
        capacite = _____;
    }

    snprintf(
        annuaire[taille].email,
        EMAIL_MAX,
        "%s",
        _____
    );

    annuaire[taille].id = _____;

    _____;

    //inclure un printf si besoin pour capacite et taille
}

void seq_free(void)
{
    _____;
    _____;
    _____;
}
```

N'oubliez pas de déclarer `seq_insert`, `seq_search` et `seq_free` dans `annuaire.h`.

Format attendu

Fonctions complétées :

Suite des capacités observée : _____

Réponse à la question 5 : _____

Exercice 3 - La recherche séquentielle

Aide

Écriture	Ce qu'elle compare
<code>a == b</code> sur deux <code>char *</code> <code>strcmp(a, b) == 0</code>	Les adresses , presque jamais ce que l'on veut Les contenus , et vaut 0 quand ils sont identiques

Attention, `strcmp` renvoie 0 quand les chaînes sont égales.

Rappel de cours pour la question 5 : un même algorithme n'a pas un coût, il en a trois, selon ce que contiennent les données.

Cas	Les données sont...	Exemple
Favorable	Les données permettent le minimum d'opérations	Recherche d'un nombre dans un tableau : le nombre recherché est le premier élément .
Moyen	Les données sont quelconques	Recherche d'un nombre dans un tableau : le nombre recherché est quelque part au milieu du tableau.
Défavorable	Les données nécessitent le maximum d'opérations	Recherche d'un nombre dans un tableau : le nombre recherché est le dernier élément ou n'est pas présent .

Consigne

1. Complétez `seq_search`, qui renvoie `true` si l'e-mail est présent et `false` sinon.
2. Testez sur un annuaire contenant cinq adresses : trois recherches qui doivent réussir, deux qui doivent échouer.
3. Testez sur un annuaire **vide**, sans aucune insertion. Le programme doit répondre `false` sans planter.
4. Remplacez `strcmp(...) == 0` par `... == email`. Le programme compile-t-il ? Que renvoie la recherche ? Notez précisément ce qui se passe.
5. Question : combien de comparaisons `seq_search` effectue-t-elle sur un annuaire de `n` utilisateurs, dans le cas favorable, dans le cas moyen, puis dans le cas défavorable ? Décrivez à chaque fois quelle donnée produit ce cas.

Modèle à compléter

```
bool seq_search(const char *email) {  
    for (int i = 0; i < _____; i++)  
        if (_____)  
            return true;  
    return _____;  
}
```

Format attendu

Fonction complétée :

Recherche	Attendu	Obtenu
Une adresse présente	true	
Une adresse absente	false	
Sur annuaire vide	false	

Observation de la question 4 : _____

Réponse à la question 5 : _____

Exercice 4 - Calculer une adresse plutôt que chercher

Aide

Une fonction de hachage transforme une chaîne en un nombre. On ramène ensuite ce nombre à un indice valide avec le modulo. Essayer d'expliquer avec vos mots ce qu'est une fonction d'hachage (*hint*: ça peut servir pour le quiz de fin de cours...)

```
unsigned long h = hachage(email);
unsigned long indice = h % TAILLE_TABLE;
```

Le type doit être `unsigned long`, jamais `long`. Avec un type signé, le dépassement de capacité produit une valeur négative, et le modulo donne alors un indice négatif, donc un accès hors du tableau (remember : https://en.wikipedia.org/wiki/Ariane_flight_V88).

Consigne

1. Créez `hachage.c` et écrivez la fonction `hachage` à partir du modèle, qui implémente l'algorithme djb2 (si vous avez envie d'en savoir plus : <https://mojoauth.com/security-guides/bernsteins-hash-djb2-in-python#how-the-djb2-hash-algorithm-works>).
2. Dans un `main`, affichez l'indice obtenu pour les cinq adresses `alice@mail.com`, `bob@mail.com`, `carole@mail.com`, `david@mail.com`, `eve@mail.com`, avec `TAILLE_TABLE` valant 1024.
3. Affichez l'indice de `alice@mail.com` trois fois de suite. Que constatez-vous, et pourquoi est-ce indispensable ?
4. Comparez les indices de `user1@mail.com` et `user2@mail.com`. Sont-ils voisins ?
5. Remplacez `unsigned long` par `int` dans la fonction **et** dans la variable qui reçoit l'indice, puis réaffichez les indices des cinq adresses. Deux d'entre elles changent de comportement : lesquelles, et que vaut alors leur indice ? Que se passerait-il si on utilisait cet indice pour accéder au tableau ?
6. Question : deux adresses différentes peuvent-elles donner le même indice ? Si oui, est-ce un défaut de la fonction ?

Modèle à compléter

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "annuaire.h"

#define TAILLE_TABLE 1024

unsigned long hachage(const char *email) {
    unsigned long h = 5381;
    int c;
    while ((c = (unsigned char)*email++))
        h = _____;
    return h;
}
```

Indice : djb2 multiplie l'accumulateur par 33 et ajoute le caractère courant.

Format attendu

Fonction complétée :

Adresse	Indice obtenu
alice@mail.com	
bob@mail.com	
carole@mail.com	
david@mail.com	
eve@mail.com	

Observation de la question 3 : _____

Observation de la question 4 : _____

Observation de la question 5 : _____

Réponse à la question 6 : _____

Exercice 5 - La table de hachage avec chaînage

Chaque alvéole de la table contient une liste chaînée. Deux adresses qui tombent sur le même indice cohabitent dans la même chaîne.

```
typedef struct Node {
    char email[EMAIL_MAX];
    int id;
    struct Node *next;
} Node;
```

```
static Node *table[TAILLE_TABLE] = { NULL };
```

L'insertion se fait **en tête** de chaîne, ce qui coûte un temps constant. L'ordre des utilisateurs dans une alvéole n'a aucune importance.

Ordre des deux lignes	Effet
<code>n->next = table[i];</code> puis <code>table[i] = n;</code>	Le nouveau nœud précède la chaîne existante
L'inverse	La chaîne existante est perdue, et la mémoire avec

Consigne

1. Complétez `hash_insert`, `hash_search` et `hash_free`.
2. Testez avec les mêmes cinq adresses que l'exercice 3, plus les deux recherches qui doivent échouer, plus le cas de l'annuaire vide.
3. Inversez volontairement les deux lignes de l'insertion en tête. Insérez trois adresses tombant dans la même alvéole, puis recherchez-les. Que se passe-t-il ?
4. Question : `hash_free` doit libérer chaque nœud de chaque chaîne. Pourquoi faut-il mémoriser `n->next` dans une variable **avant** d'appeler `free(n)` ?

Modèle à compléter

```
void hash_insert(const char *email, int id) {
    unsigned long i = hachage(email) % TAILLE_TABLE;
    Node *n = malloc(sizeof(Node));
    if (n == NULL) { perror("malloc"); exit(EXIT_FAILURE); }
    snprintf(n->email, EMAIL_MAX, "%s", email);
    n->id = id;
    n->next = _____;
    table[i] = _____;
}

bool hash_search(const char *email) {
    unsigned long i = _____;
```

```

    for (Node *n = _____; n != NULL; n = _____)
        if (_____)
            return true;
    return false;
}

void hash_free(void) {
    for (int i = 0; i < TAILLE_TABLE; i++) {
        Node *n = table[i];
        while (n != NULL) {
            Node *suiv = _____;
            free(n);
            n = _____;
        }
        table[i] = NULL;
    }
}

```

Format attendu

Fonctions complétées :

Recherche	Attendu	Obtenu
Une adresse présente	true	
Une adresse absente	false	
Sur annuaire vide	false	

Observation de la question 3 : _____

Réponse à la question 4 : _____

Exercice 6 - Synthèse : un banc de test commun

Situation

Les deux approches doivent donner **exactement** le même résultat sur les mêmes données. Un banc de test commun est la seule façon de s'en assurer, et il servira à chaque nouvelle approche ajoutée pendant le module.

Consigne

Écrivez `test.c` qui :

1. vérifie que les deux approches répondent **false** sur un annuaire vide ;
2. insère les mêmes cinq utilisateurs dans les deux structures ;
3. vérifie que les cinq adresses sont trouvées par les deux approches ;
4. vérifie que deux adresses absentes ne sont trouvées par aucune des deux ;
5. vérifie qu'une adresse dont seule la casse diffère (Alice@mail.com) n'est **pas** trouvée ;
6. affiche pour chaque test [OK] ou [ECHEC], puis le compte final ;
7. libère toute la mémoire et sort avec le code 0 si tous les tests passent, 1 sinon.

Indices : écrivez une fonction `verifier(const char *titre, bool obtenu, bool attendu)` qui compare, affiche et met à jour deux compteurs. Pensez à `seq_free()` et `hash_free()` avant le `return`.

Vérification finale

```

make clean && make
./test ; echo "code de retour : $?"

```

La compilation ne doit produire **aucun avertissement**.

Format attendu

Fichier `test.c` :

Sortie obtenue :

Code de retour : _____

À rendre

- Les fichiers `annuaire.h`, `sequentiel.c`, `hachage.c`, `test.c` et `Makefile`, compilant sans avertissement.
- La grille d'analyse de l'exercice 1.
- Les tableaux de résultats et les réponses aux questions des exercices 0 à 5.
- La sortie du banc de test de l'exercice 6.